

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут

Кафедра математичних методів захисту інформації

Звіт до лабораторної № 3
за темою:
Реалізація смарт-контракту в мережі Solana

Виконав:
ФІ-52мн Ісаченко Н. С.

м. Київ
2026р.

Мета: Отримання навичок роботи із децентралізованими додатками та оцінка безпеки інформації при їх функціонуванні

Завдання: розробка децентралізованого додатку захисту інтелектуальної власності цифрового контенту на мережі Solana

Мета: Отримання навичок роботи із децентралізованими додатками та оцінка безпеки інформації при їх функціонуванні.

Завдання: розробка децентралізованого додатку захисту інтелектуальної власності цифрового контенту на мережі Solana.

—

Теоретичні відомості: Особливості архітектури Solana та відмінності від Ethereum

Платформа Solana є високопродуктивним блокчейном третього покоління, створеним для забезпечення масштабованості без використання технологій шардингу чи архітектур другого рівня (L2). На відміну від більшості класичних децентралізованих систем, Solana фокусується на оптимізації пропускну здатності на рівні виконання транзакцій та консенсусу.

Ключові відмінності архітектури Solana та Ethereum

Для глибшого розуміння специфіки розробки децентралізованих додатків (dApps) варто порівняти архітектурні парадигми Solana та Ethereum:

1. Розділення логіки та стану (State-Logic Separation):
 - Ethereum (EVM): Смарт-контракти об'єднують у собі як код (логіку), так і стан (змінні, баланси). Кожен контракт має власне сховище даних (storage).
 - Solana: Програми (Programs) є повністю незмінними («stateless») — вони містять виключно виконуваний код і не зберігають дані всередині себе. Усі дані записуються в окремі сутності — акаунти (Accounts). Програма лише зчитує дані з переданих їй акаунтів і модифікує їх, якщо має на це відповідні права.
2. Модель виконання транзакцій:
 - Ethereum: Транзакції виконуються послідовно в рамках Віртуальної Машини Ethereum (EVM). Це обмежує масштабованість швидкістю одного ядра процесора ноди.
 - Solana: Завдяки рушію Sealevel забезпечується паралельне виконання транзакцій. Якщо транзакції не взаємодіють з одними й тими самими акаунтами, вони обробляються одночасно на різних ядрах процесора. Для цього клієнт зобов'язаний заздалегідь явно передати повний список акаунтів, які будуть зчитуватись чи модифікуватись.
3. Механізм консенсусу:
 - Ethereum: Використовує класичний Proof-of-Stake (PoS), де час формування блоку становить близько 12 секунд.
 - Solana: Поєднує Proof-of-Stake із унікальним механізмом Proof-of-History (PoH). PoH є логічним годинником (криптографічною

послідовністю SHA-256), що дозволяє нодам верифікувати час і черговість подій без необхідності постійної комунікації між собою. Це знижує затримку формування блоку до 400 мілісекунд.

4. Мова програмування:

- Ethereum: Переважно Solidity (спеціалізована високорівнева мова).
- Solana: Rust, C, C++ (низькорівневі мови із суворою типізацією та керуванням пам'яттю), що компілюються у байт-код BPF/SBF (Solana Bytecode Format).

—

Архітектура розробленого децентралізованого додатку

Для реалізації системи захисту інтелектуальної власності (IP) було використано фреймворк Anchor (де-факто стандарт для розробки безпечних програм на Solana). Додаток дозволяє авторам реєструвати права на цифровий контент за допомогою його унікального криптографічного хешу (`content_hash`), видавати тимчасові або безстрокові ліцензії іншим користувачам, а також очищувати застарілі записи.

Структура збереження даних (`state.rs`)

Оскільки програми в Solana не мають стану, дані структуровано у вигляді двох типів акаунтів:

- `IpRecord`: Зберігає інформацію про первинну реєстрацію інтелектуальної власності. Містить публічний ключ власника (`owner`), хеш файлу (`content_hash`), назву об'єкта (`title`), часові мітки створення (`created_at`) і завершення дії прав (`expires_at`), а також значення `bump` для валідації адреси.
- `LicenseRecord`: Зберігає інформацію про видану ліцензію. Містить посилання на оригінальний акаунт IP (`ip_record`), гаманець отримувача (`licensee`) та термін дії ліцензії.

Логіка інструкцій та бізнес-логіка програми

Програма підтримує три основні операції, реалізовані у модулях `instructions.rs` та `lib.rs`:

1. **`register_ip`** (Реєстрація об'єкта): Створює акаунт `IpRecord`. Адреса акаунта є детермінованою програмно-керованою адресою (PDA), яка виводиться з префіксу `ip_record`, ключа власника та хешу контенту. Це гарантує, що один і той самий власник не може зареєструвати один і той самий хеш двічі.
2. **`grant_license`** (Надання ліцензії): Створює акаунт `LicenseRecord`. Адреса прив'язується до ключів самого запису IP та ліцензіата. Інструкція містить макрос `has_one = owner`, який на рівні середовища Anchor

перевіряє, чи є підписант транзакції (Signer) дійсним власником, вказаним в `IpRecord`.

3. **delete_ip** (Видалення запису): Закриває акаунт `IpRecord` і повертає заблоковані кошти за оренди пам'яті (`Rent`) на гаманець `receiver`. У тілі функції реалізовано унікальну бізнес-логіку: якщо запис видаляє сам власник, операція дозволена в будь-який момент. Якщо ж ініціатором є сторонній гаманець, видалення можливе лише тоді, коли термін дії запису (`expires_at`) минув.

—

Розгортання (Deploy) програм у мережі Solana

Процес деплою в Solana суттєво відрізняється від EVM-мереж через механізм управління пам'яттю та оновлюваність коду.

Послідовність кроків деплою

1. Генерація ключів програми: Перед компіляцією розробник генерує унікальну пару ключів для програми (Keypair). Публічний ключ цієї пари стає `Program ID`. Він прописується в `lib.rs` у макросі `declare_id!()` та в конфігураційному файлі проекту `Anchor.toml`.
2. Компіляція (Build): Команда `anchor build` запускає компілятор Rust, який створює оптимізований файл байт-коду із розширенням `.so` (Shared Object) та генерує файл IDL (Interface Definition Language) у форматі JSON, який описує структуру програми для взаємодії з веб-інтерфейсами.
3. Ініціалізація та завантаження нативного коду (Deploy): Виклик команди `anchor deploy` виконує під капотом складну послідовність транзакцій системи:
 - Створюється спеціальний буферний акаунт (Buffer Account), куди частинами завантажувється скомпільований байт-код `.so` (через обмеження на максимальний розмір однієї транзакції в Solana — 1232 байти).
 - Після повного завантаження створюється програмний акаунт (Program Account) та акаунт даних програми (Program Data Account). Буфер копіюється в Program Data Account, і прапорець акаунта встановлюється як виконуваний (`executable = true`).

Економіка деплою (Rent)

Для зберігання байт-коду програми (який зазвичай займає від 100 до 400 КБ) на блокчейні необхідно заблокувати значну кількість SOL як забезпечення оренди (`Rent-exempt balance`). Ці кошти повертаються розробнику лише у випадку повного видання/закриття програми з блокчейну за допомогою CLI-команди `solana program close`.

Аналіз безпеки інформації розробленого додатку

Аналіз вихідного коду програми дозволяє оцінити рівень захищеності децентралізованого додатку від поширених векторів атак у мережі Solana.

1. Валідація акаунтів та захист від підміни (Account Substitution)

У багатьох нативних програмах на Solana розробники часто забувають перевіряти, чи належить переданий акаунт поточній програмі.

Фреймворк Anchor та архітектура нашого коду вирішують це автоматично через використання PDA (Program Derived Addresses):

```
#[account(
    seeds = [b"ip_record", ip_record.owner.as_ref(), content_hash.as_ref()],
    bump = ip_record.bump,
    close = receiver
)]
pub ip_record: Account<'info, IpRecord>,
```

Під час виклику інструкції `DeleteIp` Anchor самостійно бере адресу власника з переданого акаунта `ip_record.owner`, обчислює детермінований хеш із сідами і звіряє його з адресою акаунта, наданого клієнтом. Якщо зломисник спробує підставити фейковий акаунт, верифікація сідів (`seeds`) та бампа (`bump`) завершиться помилкою виконання.

2. Перевірка повноважень підписанта (Missing Signer Check)

Операції реєстрації та ліцензування критично залежать від автентифікації користувача. В інструкціях `RegisterIp` та `GrantLicense` акаунт власника типізовано як `Signer<'info>`:

```
pub owner: Signer<'info>,
```

Це гарантує, що транзакція підписана закритим ключем цього гаманця. Без дійсного криптографічного підпису транзакція буде відхилена на рівні рантайму Solana ще до потрапляння в логіку смарт-контракту.

3. Авторизація доступу при ліцензуванні (Ownership Verification)

В інструкції `GrantLicense` застосовано критично важливий атрибут безпеки:

```
#[account(
    seeds = [b"ip_record", owner.key().as_ref(), content_hash.as_ref()],
    bump = ip_record.bump,
    has_one = owner,
)]
pub ip_record: Account<'info, IpRecord>,
```

Обмеження `has_one = owner` виконує строгу перевірку: поле `owner` всередині збережених даних акаунта `IpRecord` повинно збігатися з відкритим ключем акаунта `owner` (який є підписантом `Signer`). Це повністю

унеможлиблює атаку, при якій сторонній користувач намагався б видати ліцензію на чужу інтелектуальну власність.

4. Аналіз логіки часових обмежень та економічних стимулів (Time-lock & Rent Reclamation)

Функція `delete_ip` містить умовний оператор контролю доступу:

```
if current_wallet != ip_record.owner {  
    require!(ip_record.expires_at > 0, IpError::CannotDeleteYet);  
    require!(clock.unix_timestamp > ip_record.expires_at,  
IpError::CannotDeleteYet);  
}
```

Позитивний аспект: Власник може видалити свій запис у будь-який момент. Якщо ж термін дії прав закінчився (`clock.unix_timestamp > ip_record.expires_at`), будь-який сторонній користувач (`current_wallet`) отримує право виконати цю інструкцію. Економічний нюанс (Вразливість дизайну / Стимул): Оскільки атрибут акаунта вказує `close = receiver`, кошти оренди (SOL) повертаються на адресу `receiver`, яка передається як довільний параметр транзакції:

```
/// CHECK: Отримувач поверненої ренти (зазвичай збігається з власником IP)  
pub receiver: AccountInfo<'info>,
```

Оскільки `receiver` ніяк не валідується всередині коду (немає перевірки, що `receiver == ip_record.owner`), будь-який сторонній користувач, який ініціює очищення простроченого запису, може вказати свій власний гаманець як `receiver` і забрати собі накопичену ренту (SOL). З одного боку, це діє як економічний стимул (сміттяр / garbage collector) для очищення блокчейну від застарілих даних. З іншого боку, якщо оригінальний власник IP планував повернути свої кошти за оренду після завершення терміну дії, сторонній бот може випередити його (front-run транзакцію) і привласнити SOL. Для підвищення безпеки логічно було б замінити `close = receiver` на жорстку відправку коштів на `ip_record.owner` або розподіляти ренту між власником та ліквідатором.

Висновки

У ході виконання лабораторної роботи було успішно розроблено та проаналізовано децентралізований додаток для захисту інтелектуальної власності цифрового контенту на базі блокчейну Solana з використанням фреймворку Anchor та мови програмування Rust.

Було детально розглянуто архітектурну специфіку Solana, зокрема її «stateless» модель, яка розділяє виконуваний код (програми) та сховище даних (акаунти). Порівняльний аналіз із платформою Ethereum показав переваги паралельного виконання транзакцій за допомогою рушія Sealevel та консенсусу Proof-of-History, що дозволяє суттєво оптимізувати швидкість роботи порівняно з EVM-системами.

Проведений аналіз безпеки розробленого смарт-контракту підтвердив високу надійність архітектури завдяки автоматичній перевірці підписів (Signer), строгому детермінованому виведенню адрес акаунтів через механізм PDA (seeds та bump), а також впровадженню обмеження has_one. Виявлена особливість у логіці повернення ренти при видаленні прострочених записів сторонніми користувачами продемонструвала важливість ретельного проектування токеноміки та перевірки адрес отримувачів (receiver) для запобігання несанкціонованому перехопленню коштів ботами-ліквідаторами. Додаток повністю виконує поставлені завдання та демонструє практичні навички безпечної розробки у середовищі Web3.